

# CYBERSECURITY & ETHICAL HACKING

## TASK 4 – WEB APPLICATION VULNERABILITY ASSESSMENT

RabTech Academy — Cybersecurity Internship

**Assessor: Pritam Bose**  
*OWASP Top 10 – Practical Assessment Report*

|                        |                                          |
|------------------------|------------------------------------------|
| <b>Target</b>          | OWASP Juice Shop – Local Sandbox         |
| <b>Assessment Type</b> | Web Application Vulnerability Assessment |
| <b>Tools</b>           | Burp Suite, Nmap, Browser                |
| <b>Evidence</b>        | SS01, SS02, SS03A, SS03B, SS04           |
| <b>Scope</b>           | Authorized local training environment    |

## 1. Executive Summary

This assessment was performed against a locally hosted OWASP Juice Shop instance in an authorized sandbox. The supplied evidence demonstrates reconnaissance, SQL Injection, DOM-based XSS, and an object-level authorization weakness consistent with BOLA/IDOR testing. Burp Suite was used for HTTP request/response analysis and Nmap for basic reconnaissance.

## 2. Scope and Methodology

- Target: locally hosted OWASP Juice Shop (localhost / 127.0.0.1).
- Testing was limited to the authorized local training environment.
- Nmap was used for basic service discovery.
- Burp Suite HTTP history/Repeater and the browser were used to inspect and validate application behavior.
- Only the supplied screenshots are treated as demonstrated proof.

## 3. Evidence Summary

| Evidence | Area           | Observed Result                                                | Status       |
|----------|----------------|----------------------------------------------------------------|--------------|
| SS05     | Reconnaissance | TCP/3000 open; HTTP response associated with Juice Shop.       | Demonstrated |
| SS01     | SQL Injection  | Crafted login input returned HTTP 200 and authentication data. | Demonstrated |
| SS02     | DOM XSS        | Juice Shop displayed successful DOM XSS challenge completion.  | Demonstrated |
| SS03A    | BOLA/IDOR      | Original basket object retrieved as baseline.                  | Baseline     |
| SS03B    | BOLA/IDOR      | Changing object ID returned another object's data.             | Demonstrated |

## 4. SQL Injection

**Category:** OWASP Injection / SQL Injection **Severity:** High

### Observation

The Burp Suite evidence shows a POST request to /rest/user/login where the email parameter contains SQL syntax. The server returned HTTP 200 OK and an authentication object containing a token and user information.

### Potential Impact

Depending on the underlying query, SQL Injection can permit authentication bypass, unauthorized data access, or manipulation of database operations.

### Remediation

Use parameterized queries/prepared statements. Never concatenate untrusted input into SQL. Apply input validation and least-privilege database accounts.

## Defensive Code Example

// Unsafe:

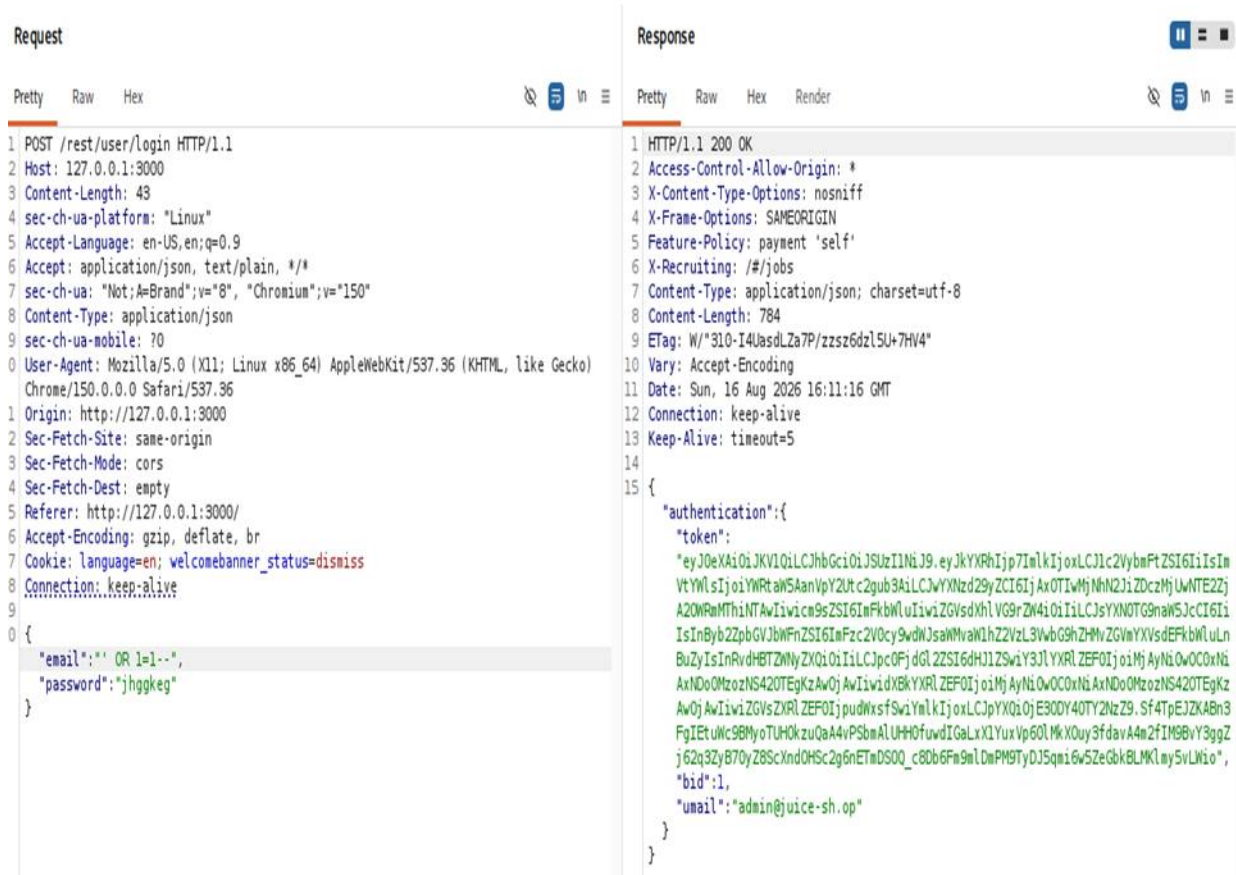
```
const query = `SELECT * FROM Users WHERE email = '${email}'`;
```

// Safer:

```
const query = 'SELECT * FROM Users WHERE email = ?';
```

```
const [users] = await db.execute(query, [email]);
```

## Evidence



SS01 – Burp Suite request/response showing SQL Injection input and successful response.

## 5. DOM-Based Cross-Site Scripting (XSS)

**Category:** Cross-Site Scripting **Severity:** Medium/High

### Observation

The supplied Juice Shop screenshot shows successful completion of the DOM XSS challenge. The browser URL contains the crafted input and the application displays a green success notification.

### Potential Impact

DOM XSS can cause attacker-controlled script content to execute in a victim's browser in the application's security context, potentially enabling unauthorized actions or data exposure.

## Remediation

Avoid unsafe DOM sinks. Prefer safe text rendering and context-aware output encoding. If HTML is genuinely required, sanitize it and use an appropriate Content Security Policy.

## Defensive Code Example

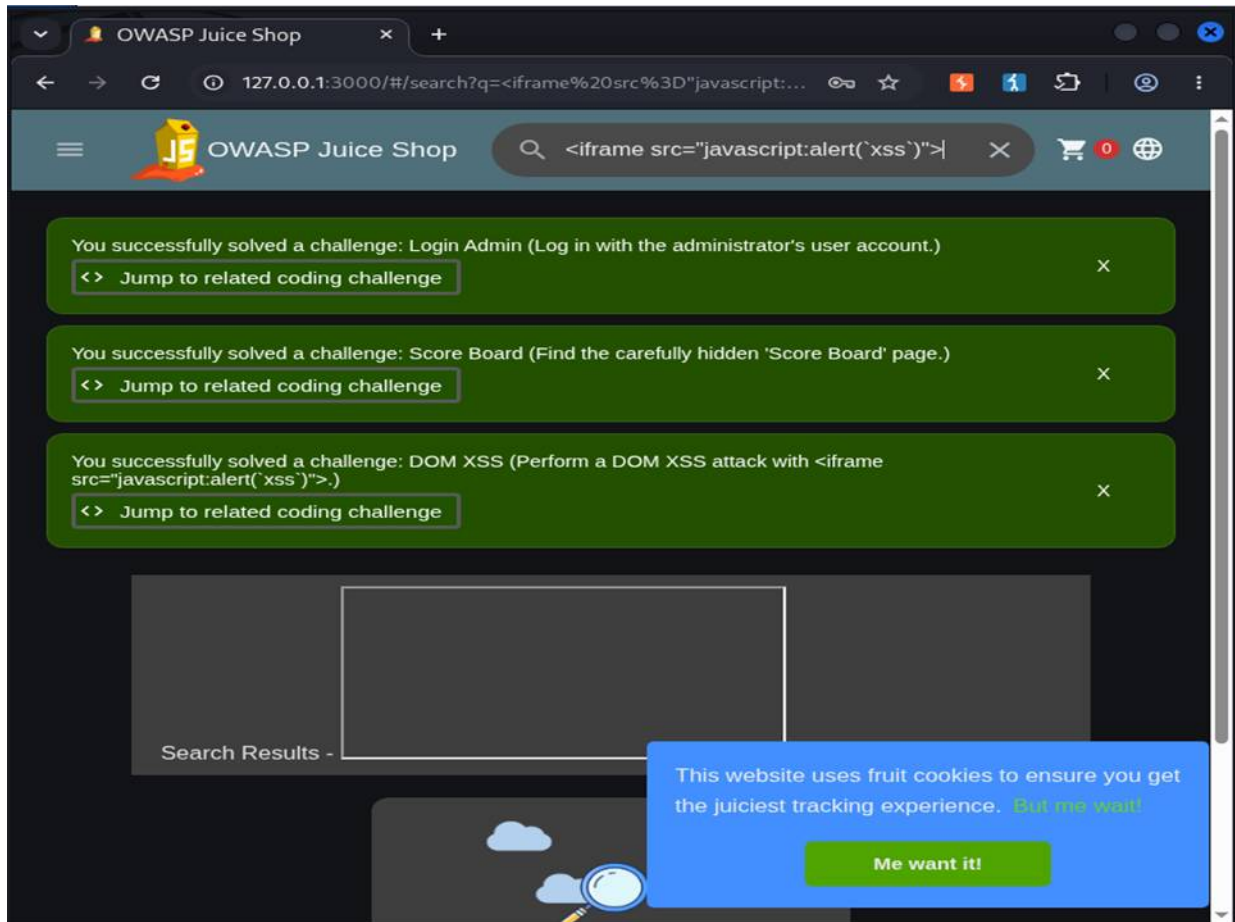
// Safe React rendering:

```
<p>{userInput}</p>
```

// Avoid unsafe raw HTML unless deliberately sanitized:

```
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

## Evidence



SS02 – Juice Shop evidence showing successful DOM XSS challenge completion.

## 6. Broken Object Level Authorization (BOLA / IDOR)

**Category:** Broken Access Control **Severity:** High

### Observation

SS05A establishes the original basket request. SS05B shows the object identifier changed while the server still returned HTTP 200 and another basket object's data. This demonstrates insufficient object-level authorization in the lab.

## Potential Impact

An attacker may access another user's resources by changing an identifier in an API request. If write operations are also affected, the impact can include unauthorized modification or deletion.

## Remediation

Authorize every object access on the server. Verify that the authenticated user owns the requested object or has an explicitly authorized role/permission. Do not rely on the client to enforce ownership.

## Defensive Code Example

```
const basket = await Basket.findByPk(req.params.id);
```

```
if (!basket) {
  return res.status(404).json({ error: 'Basket not found' });
}
```

```
if (basket.UserId !== req.user.id && req.user.role !== 'admin') {
  return res.status(403).json({ error: 'Forbidden' });
}
```

```
return res.json(basket);
```

## Evidence

[illegible]



```

(kali@kali)-[~]
$ nmap -sV -p 3000 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 11:17 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000068s latency).

PORT      STATE SERVICE VERSION
3000/tcp   open  ppp?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nm
SF-Port3000-TCP:V=7.99%I=7%D=9/16%Time=6AAAB2FA%P=x86_64-pc-linux-gnu%(Ge
SF:tRequest,2668,"HTTP/1.1\x20200\x200K\r\nAccess-Control-Allow-Origin:\x
SF:20*\r\nX-Content-Type-Options:\x20nosniff\r\nX-Frame-Options:\x20SAMEO
SF:RIGIN\r\nFeature-Policy:\x20payment\x20'self'\r\nX-Recruiting:\x20/#/jo
SF:bs\r\nAccept-Ranges:\x20bytes\r\nCache-Control:\x20public,\x20max-age=0
SF:\r\nLast-Modified:\x20Wed,\x2016\x20Sep\x202026\x2015:12:21\x20GMT\r\nE
SF:Tag:\x20W/\x204b1-1a0aac6aad3'\r\nContent-Type:\x20text/html;\x20charse
SF:t=UTF-8\r\nContent-Length:\x209393\r\nVary:\x20Accept-Encoding\r\nDate:
SF:\x20Wed,\x2016\x20Sep\x202026\x2015:17:14\x20GMT\r\nConnection:\x20clos
SF:e\r\n\r\n!-\r\n\x20\x20\x20Copyright\x20(c)\x202014-2026\x20Bjoern\x
SF:20Kimmerich\x20the\x20OWASP\x20Juice\x20Shop\x20contributors.\r\n\x
SF:20\x20\x20SPDX-License-Identifier:\x20MIT\r\n\x20-->\r\n<!doctype\x2
SF:0html>\n<html\x20lang=\x20en\x20data-beasties-container>\n<head>\n\x20
SF:20<meta\x20charset=\x20utf-8>\n\x20\x20<title>OWASP\x20Juice\x20Shop</
SF:title>\n\x20\x20<meta\x20name=\x20description\x20content=\x20Probably\x20
SF:the\x20most\x20modern\x20and\x20sophisticated\x20insecure\x20web\x20app
SF:lication>\n\x20\x20<meta\x20name=\x20viewport\x20content=\x20width=devi
SF:ce-width,\x20initial-scale=1>\n\x20\x20<link\x20rel=\x20preconnect\x2
SF:0href=\x20htt">\n\x20(Help,2F,"HTTP/1.1\x20400\x20Bad\x20Request\r\nConnecti
SF:on:\x20close\r\n\r\n")&r(NCP,2F,"HTTP/1.1\x20400\x20Bad\x20Request\r\n
SF:Connection:\x20close\r\n\r\n")&r(HTTPOptions,EA,"HTTP/1.1\x20204\x20No
SF:20Content\r\nAccess-Control-Allow-Origin:\x20*\r\nAccess-Control-All
SF:ow-Methods:\x20GET,HEAD,PUT,PATCH,DELETE\r\nVary:\x20Access-Contro
SF:l-Request-Headers\r\nContent-Length:\x200\r\nDate:\x20Wed,\x2016\x20Sep
SF:\x202026\x2015:17:14\x20GMT\r\nConnection:\x20close\r\n\r\n")&r(RTSPReq
SF:west,EA,"HTTP/1.1\x20204\x20No\x20Content\r\nAccess-Control-Allow-Orig
SF:in:\x20*\r\nAccess-Control-Allow-Methods:\x20GET,HEAD,PUT,PATCH,POST,D
SF:ELETE\r\nVary:\x20Access-Control-Request-Headers\r\nContent-Length:\x20
SF:0\r\nDate:\x20Wed,\x2016\x20Sep\x202026\x2015:17:14\x20GMT\r\nConnectio
SF:n:\x20close\r\n\r\n")&r(RPCCheck,2F,"HTTP/1.1\x20400\x20Bad\x20Request
SF:\r\nConnection:\x20close\r\n\r\n");

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.35 seconds

```

SS04 – Nmap reconnaissance identifying TCP/3000 and the Juice Shop HTTP response.

## 8. Overall Conclusion

The supplied evidence demonstrates several common web application security weaknesses in the authorized Juice Shop training environment. Reconnaissance identified the local web service. SQL Injection evidence shows crafted input reaching the login flow and producing an authentication response. DOM XSS evidence shows successful completion of an XSS challenge. The BOLA/IDOR evidence shows that changing an object identifier resulted in access to a different object's data.

The primary remediation priorities are parameterized database queries, safe rendering/sanitization of untrusted data, and server-side authorization checks for every object. These controls must be enforced on the server because client-side controls can be bypassed.

No stored-XSS success screenshot was supplied, so this report does not claim stored XSS as demonstrated proof. Stored-XSS remediation can still be included as a defensive recommendation if required by the assignment.

## 9. References

- OWASP Foundation – OWASP Top 10: Web Application Security Risks.
- OWASP Foundation – SQL Injection Prevention Cheat Sheet.
- OWASP Foundation – Cross Site Scripting Prevention Cheat Sheet.
- OWASP Foundation – Authorization Cheat Sheet / Broken Access Control guidance.
- OWASP Juice Shop – intentionally vulnerable web application used for security training.

Prepared By PRITAM BOSE